



Rapport de projet : OCR Word Search Solver

GREINER Matthieu **TEIXEIRA** Julien **CHAMBON** Pierre-Louis **BARROS** Dylan



FIGURE 1 – Image du groupe SegFaultless

Table des matières

1	Introduction	3
2	Le groupe	4
2.1	SegFaultless	4
2.2	Les membres de l'équipe	4
3	Le projet	5
3.1	Informations globales sur le projet	5
3.2	Procédures d'exécution du projet	6
4	La répartition des tâches	6
4.1	Les caractéristiques principales du projet	6
4.2	Pour notre première soutenance	7
4.3	Répartition des tâches pour la deuxième soutenance	8
5	Aspects techniques	8
5.1	Spécifié par le cahier des charges	8
5.1.1	Environnement de développement	8
5.1.2	Options de compilation et normes de code	8
5.1.3	Gestion du dépôt git	8
5.1.4	Ressources visuelles	8
5.1.5	Exclusions du dépôt	9
5.2	Mis en place par nous mêmes	9
5.2.1	Les bibliothèques et les logiciels utilisés	9
5.2.2	Le chargement d'une image	10
5.2.3	Prétraitement de l'image	10
5.2.4	La rotation manuelle de l'image	12
5.3	Détection des clusters de lettres	13
5.3.1	Définition et structure d'un cluster	13
5.3.2	Fonction <code>detect_clusters</code>	14
5.3.3	Détection des clusters de pixels avec la méthode Flood Fill	15
5.3.4	Structure et objectifs de la fonction	15
5.3.5	Étapes détaillées de l'algorithme	15
5.3.6	Défis rencontrés et solutions apportées	16
5.4	Sauvegarde des clusters de lettres dans des dossiers distincts	16
5.4.1	Structure et objectifs de la fonction	16
5.4.2	Étapes détaillées de la fonction	17
5.5	Renommage des fichiers de clusters	18
5.5.1	Fonctionnement détaillé	18
5.5.2	Avantages de l'approche	19
5.6	Défis et observations	19
5.6.1	Défis rencontrés et solutions apportées	19
5.6.2	Résultats et observations	19
5.6.3	Défis et résultats	19

5.6.4	L'implémentation de l'algorithme de résolution d'une grille de mots cachés	20
5.6.5	Le développement d'un réseau de neurones	20
5.6.5.1	Préparation des données d'entraînement	20
5.6.5.2	Initialisation et architecture du réseau	20
5.6.5.3	Propagation avant et calcul des erreurs	21
5.6.5.4	Rétropropagation et mise à jour des poids	21
5.6.5.5	Entraînement et sauvegarde des paramètres	22
5.7	Construction et traçage des lignes reliant les mots	22
5.7.1	Extraction des coordonnées des mots	22
5.7.2	Fonction <code>run_solver</code>	22
5.7.3	Traçage des lignes	23
5.7.4	Interface graphique	23
5.7.5	Les tâches restantes	23
6	États d'avancement du projet	24
6.0.1	Chargement et prétraitement des images	24
6.1	La rotation automatique de l'image	25
6.2	La détection de la position	25
6.3	Le découpage de l'image	26
6.4	La preuve de développement d'un réseau de neurones	27
6.5	Interface graphique	27
6.5.1	Première version	27
6.5.2	Deuxième version	28
6.6	Tableau d'avancement des tâches	32
7	Conclusion	32
A	Annexes	34

1 Introduction

Dans un monde où l'automatisation est de plus en plus présente, la reconnaissance optique de caractères (OCR) s'impose comme une technologie incontournable, permettant de transformer des images de textes en données numériques exploitables. Que ce soit pour numériser des documents, extraire des informations ou faciliter des recherches dans des fichiers scannés, l'OCR est devenu un outil essentiel dans de nombreux secteurs, de l'administration aux applications mobiles en passant par les outils de recherche en entreprise. Notre équipe, SegFaultless, a entrepris de concevoir un système OCR dans le cadre de notre projet universitaire, avec pour objectif de résoudre un problème spécifique : l'identification et la résolution de grilles de mots cachés à partir d'images. Ce projet représente un défi passionnant, alliant manipulation d'images, algorithmie avancée et intelligence artificielle.

Le processus que nous avons mis en place pour atteindre cet objectif repose sur plusieurs étapes clés. La première consiste à charger l'image et supprimer les couleurs superflues pour simplifier le traitement. Nous avons déjà développé une fonction de chargement d'images aux formats courants tels que PNG et JPEG en utilisant la bibliothèque SDL. Cependant, pour une meilleure efficacité de traitement, il est également nécessaire de convertir ces images en niveaux de gris, ce qui facilite les opérations de détection de contours et de reconnaissance de motifs.

Une autre fonctionnalité essentielle déjà en place est la rotation manuelle de l'image. Notre système permet de pivoter l'image par intervalles de 90 degrés, selon les indications de l'utilisateur. Cette fonctionnalité est particulièrement utile pour aligner correctement la grille de mots cachés et garantir un traitement précis lors des étapes suivantes.

La phase suivante implique le découpage de l'image en sous-parties représentant chaque lettre. Cette tâche, bien qu'encore en cours de développement, est cruciale pour alimenter notre système OCR. En découpant chaque lettre en une image individuelle, nous faciliterons l'entrée de données pour l'algorithme de reconnaissance de caractères, améliorant ainsi la précision de l'identification des lettres dans la grille.

Notre équipe a également consacré des efforts significatifs au développement de l'algorithme de résolution de la grille de mots cachés. Ce module, accessible via la ligne de commande, utilise une approche de recherche de mots pour détecter les termes présents dans la grille. Nous avons commencé avec une solution simple, que nous prévoyons d'optimiser pour gagner en efficacité et en rapidité.

Enfin, en parallèle de notre système OCR, nous avons développé une preuve de concept d'un réseau de neurones. Bien que modeste, ce réseau a pour but de démontrer notre capacité à intégrer des techniques d'intelligence artificielle dans notre projet. Cette approche pourrait, à terme, nous permettre d'améliorer la précision de notre système OCR en utilisant l'apprentissage automatique pour affiner la reconnaissance des lettres et des mots dans la grille.

En conclusion, le projet OCR de SegFaultless est une entreprise ambitieuse qui requiert une maîtrise des techniques de traitement d'image et d'algorithmes de recherche, ainsi qu'une compréhension des réseaux de neurones. Chaque étape, qu'il s'agisse de la manipulation d'images ou de l'implémentation d'algorithmes, nous rapproche de notre objectif final : concevoir un outil capable de résoudre automatiquement des grilles de mots cachés à partir d'images. Cette expérience enrichissante nous permet d'explorer les multiples facettes de l'OCR tout en contribuant au développement de compétences précieuses pour notre avenir professionnel.

2 Le groupe

2.1 SegFaultless

Le choix du nom SegFaultless pour notre groupe est à la fois un clin d'œil technique et un engagement envers la qualité de notre travail. Inspiré du terme "Segmentation Fault" (ou "SegFault"), une erreur courante en programmation indiquant un accès mémoire non autorisé, notre nom reflète notre volonté d'éviter les erreurs critiques et de créer un projet "faultless" — sans défaut. Ce jeu de mots incarne notre ambition de dépasser les obstacles techniques tout en produisant un travail rigoureux, stable et professionnel.

2.2 Les membres de l'équipe

Le groupe **SegFaultless** est composé de quatre membres aux compétences complémentaires, chacun jouant un rôle clé dans le projet OCR Word Search Solver. Voici les membres de l'équipe :

— **Julien :**

Je m'appelle Julien et je suis passionné par le développement logiciel et l'intelligence artificielle. Ces domaines m'inspirent énormément car ils nous permettent de résoudre des problèmes complexes de manière créative tout en ouvrant la voie à des avancées technologiques qui ont un impact sur la vie quotidienne. Dans ce projet, les aspects techniques qui m'ont le plus attiré étaient la programmation et l'optimisation des algorithmes. Ces domaines me permettront d'améliorer mes compétences en rédaction de code efficace, en construction de projets complexes et en résolution de problèmes en temps réel. Travailler en équipe est pour moi une grande source de motivation. Non seulement cela m'a donné une nouvelle perspective sur les projets, mais cela m'a également offert un environnement de travail dynamique où chacun pouvait apporter ses forces et ses idées. Nous avons décidé de travailler ensemble car nous partageons la même ambition.

— **Pierre Louis :**

J'ai découvert l'informatique à l'âge de 9 ans grâce au jeu vidéo Minecraft. J'ai très vite eu envie de comprendre ce qu'il se passait derrière pour qu'un jeu comme Minecraft puisse fonctionner. C'est à l'âge de 12 ans que j'ai commencé la programmation dans le but de développer un serveur minecraft avec différents mini-jeux. Depuis je n'ai jamais quitté la programmation. Durant mon parcours, j'ai hésité entre médecine et informatique, aujourd'hui moi but et de combiner ces deux secteurs en faisant de l'informatique en lien avec le corps de la Santé, je suis particulièrement intéressé par les réseaux de neurone d'analyse d'image qui adapté à la médecine peuvent permettre par exemple de détecter des cancers à des stades très précoces. Ce projet d'OCR est donc extrêmement important pour moi et je suis prêt à me donner à 200% pour mener à bien ce projet.

— **Dylan :**

Depuis que j'ai découvert le développement de jeux, je suis devenu passionné par la technologie et ses applications innovantes, notamment dans des domaines comme l'intelligence artificielle et l'analyse d'images. Cet intérêt m'a conduit à explorer des sujets comme les réseaux de neurones, dont les capacités d'analyse sont particulièrement prometteuses, notamment en médecine. Par exemple, je suis fasciné par la manière dont les réseaux neuronaux peuvent contribuer à la détection précoce de maladies graves, comme le cancer, où une détection rapide peut faire une réelle différence pour les patients.

Bien que je n'aie pas encore eu l'occasion de travailler sur un projet de reconnaissance optique de caractères (OCR), j'ai hâte de m'y mettre. Pour moi, ce sera une occasion en or d'approfondir mes connaissances en traitement d'image et en IA. Je suis prêt à m'investir pleinement et à relever tous les défis, afin de réussir ce projet et de développer des compétences essentielles dans ces domaines.

Bien que je n'aie pas encore eu l'occasion de travailler sur un projet de reconnaissance optique de caractères (OCR), j'ai hâte de m'y mettre. Pour moi, ce sera une occasion en or d'approfondir mes connaissances en traitement d'image et en IA. Je suis prêt à m'investir pleinement et à relever tous les défis, afin de réussir ce projet et de développer des compétences essentielles dans ces domaines.

— **Matthieu :**

Passionné par les aspects fondamentaux de la programmation en C, j'ai trouvé dans l'OCR une application concrète qui met à l'épreuve les compétences acquises lors de ma formation à l'EPITA. Ce projet me permet de manipuler des concepts clés en traitement d'image et en gestion mémoire, des domaines techniques que j'ai particulièrement explorés et qui m'enthousiasment. Ces compétences, appliquées ici dans le cadre d'un projet de groupe, renforcent mes bases pour des applications en vision par ordinateur et en reconnaissance de motifs, des domaines qui me motivent particulièrement.

Sur le plan personnel, l'OCR m'offre une opportunité unique de collaborer avec une équipe que j'apprécie pour son professionnalisme et son ambition. Nous avons décidé de travailler ensemble car chacun apporte des compétences spécifiques et une perspective complémentaire, ce qui rend le travail de groupe à la fois stimulant et enrichissant. Cette expérience nous permet non seulement de progresser techniquement, mais aussi de développer des compétences humaines essentielles, comme la communication et la gestion collaborative d'un projet complexe.

Chaque membre apporte son expertise et sa motivation pour mener à bien ce projet ambitieux, garantissant que toutes les étapes, de l'analyse des images à la résolution de la grille de mots, soient exécutées avec rigueur et efficacité.

3 Le projet

3.1 Informations globales sur le projet

Le projet de reconnaissance optique de caractères (OCR) a pour objectif de développer un système capable de détecter et de lire des lettres et des mots à partir d'images de grilles de mots cachés. Ce projet est réalisé en langage C et utilise la bibliothèque SDL pour le traitement et l'affichage des images.

Les principales fonctionnalités du projet incluent :

- Chargement d'images dans des formats courants (PNG, JPEG).
- Prétraitement des images pour améliorer la lisibilité (conversion en niveaux de gris, suppression des couleurs).
- Détection de la grille et des lettres.
- Découpage de l'image pour extraire chaque lettre en tant qu'image individuelle.
- Implémentation d'un algorithme de résolution de grilles de mots cachés.
- Démonstration d'un mini réseau de neurones pour une tâche logique simple.

Le projet vise également à respecter des contraintes de performance et de compatibilité, en s'assurant qu'il fonctionne sur les systèmes NixOS et MacOS. Les résultats seront présentés sous forme d'une interface utilisateur simple permettant de visualiser les grilles et les mots détectés.

3.2 Procédures d'exécution du projet

Pour exécuter le projet OCR, plusieurs étapes doivent être suivies :

1. Installation des dépendances :

- Assurez-vous d'avoir la bibliothèque SDL installée sur votre système. Vous pouvez l'installer via le gestionnaire de paquets Nix ou MacOS.

2. Préparation des images :

- Placez les images que vous souhaitez traiter dans le dossier `data/examples` du projet. Assurez-vous que les images sont dans un format compatible (PNG, JPEG).

3. Compilation du code :

- 4. Utilisez le Makefile fourni pour compiler le projet. Exécutez la commande suivante dans le terminal :
 - `make ocrgr` ou `make ocrb` pour compiler uniquement le chargement de l'image avec soit le traitement avec nuances de gris soit le traitement noir et blanc (par dessus les nuances de gris).
 - `make solver` pour compiler le solver manuel.
 - `make neural` pour compiler la fonction logique.
 - Et aussi `make all` si l'on veut tout compiler à la fois.

5. Exécution de l'application :

- Une fois la compilation réussie, exécutez le programme avec la commande suivante, en spécifiant le fichier image à traiter : `./bin/ data/examples/<image_file>`
- Pour le solver ca sera, `./bin grid mot voulu`
- Pour le réseau de neurones, `./bin neural_test train`, vous allez pouvoir constater l'apprentissage du réseau de neurones avec la base de données.
- Pour l'interface graphique la commande nécessaire sera : `ocr_gui`.

6. Affichage des résultats :

- Après le traitement, dans le terminal seront affichés des informations comme la rotation automatique de la grille mais aussi les informations sur la fonction logique et le réseau de neurones et pour le chargement de l'image une fenêtre apparaîtra pour montrer l'image avec les différents niveaux de gris.

Ces étapes assurent que le projet est exécuté de manière fluide, permettant aux utilisateurs de tirer pleinement parti de ses fonctionnalités de reconnaissance de caractères.

4 La répartition des tâches

4.1 Les caractéristiques principales du projet

Le projet OCR se concentre sur la détection de mots dans des images de grilles de mots cachés. Les principales caractéristiques du projet incluent :

- Chargement d'images dans des formats variés (PNG, JPEG) pour un traitement ultérieur.
- Prétraitement des images, y compris la conversion en niveaux de gris pour simplifier l'analyse.
- Détection et extraction de la grille de mots, des lettres et des mots de la liste à l'aide d'algorithmes adaptés.
- Résolution des grilles de mots cachés par un module dédié, utilisant des méthodes de recherche optimisées.
- Implémentation d'une preuve de concept pour un mini réseau de neurones, illustrant une approche d'apprentissage simple.

4.2 Pour notre première soutenance

Pour notre première soutenance, nous avons réalisé plusieurs avancées significatives :

- **Chargement des images :** Nous avons implémenté une fonction pour charger des images (formats PNG et JPEG) en utilisant la bibliothèque SDL. Cette étape est essentielle pour préparer le traitement des images.
- **Rotation manuelle de l'image :** Nous avons développé une fonction qui permet de faire pivoter l'image manuellement en capturant l'entrée de l'utilisateur pour ajuster l'angle. Cela facilite le traitement ultérieur des images qui peuvent être orientées incorrectement.
- **Détection de contours :** Nous avons commencé à travailler sur la détection de contours, bien que cette fonctionnalité reste à finaliser. L'objectif est de distinguer la grille et les lettres pour une meilleure précision dans la reconnaissance.
- **Découpage de l'image :** Nous avons prévu de découper chaque lettre détectée et de l'enregistrer en tant qu'image individuelle, ce qui facilitera l'entrée pour l'étape OCR.
- **Module de résolution de mots :** Nous avons réussi à créer un module de ligne de commande pour parcourir la grille et détecter les mots de la liste. Ce module a été conçu avec un algorithme simple de recherche de mots, qui sera optimisé dans les prochaines étapes.
- **Preuve de concept du réseau de neurones :** Nous avons également développé une preuve de concept pour un mini réseau de neurones, montrant comment il peut apprendre à réaliser des fonctions logiques simples.

4.3 Répartition des tâches pour la deuxième soutenance

Voici le tableau présentant la répartition des tâches dans notre groupe de projet. Il faut aussi noter que ces tâches ne sont pas vraiment respectives sachant que tout le monde s'entraidait sur tous les sujets.

	Matthieu	Dylan	Julien	Pierre-Louis
Rotation automatique	X		X	
Prétraitements				X
Détection de la grille et des éléments		X		
Segmentation de l'image		X		
Module de résolution des mots d'une grille		X		
Développement du réseau de neurones	X			
Compilation Makefiles	X		X	X
Interface graphique		X		
Rapport de soutenance	X	X	X	X

TABLE 1 – Tableau détaillé présentant la répartition des tâches pour la deuxième soutenance.

5 Aspects techniques

5.1 Spécifié par le cahier des charges

5.1.1 Environnement de développement

Pour l'environnement de développement nous avons particulièrement codé sur WSL qui est une machine virtuelle simulant une distro Linux ou particulièrement Ubuntu dans notre cas et nous avons utilisé Visual Studio Code pour gérer les Makefiles et les fichiers C. De plus nous avons aussi testés entièrement notre code sur les machines des salles machines en mode prépa pour s'assurer du bon fonctionnement intégral du projet sous NixOS.

5.1.2 Options de compilation et normes de code

Les options de compilation indiquées par le cahier de charges sont au moins de compiler avec les flags - Wall -Wextra. De plus le code est limité à 80 caractères par ligne pour faciliter la lecture de celui-ci.

5.1.3 Gestion du dépôt git

Le dépôt Git doit contenir absolument un Makefile final avec les commandes `all` et `clean` et le code nécessaire à la compilation du projet. Le dépôt devra contenir aussi les 2 rapports de soutenances à la racine du dépôt et étant intitulé : `rapport_n°.pdf`. De plus, il devra absolument posséder un fichier `AUTHORS` et `README`.

5.1.4 Ressources visuelles

Le dépôt GIT devra aussi contenir quelques images qui pourront servir aux démonstrations des différentes parties. Il faudra également un jeu d'images pour l'apprentissage de votre réseau de neurone.

5.1.5 Exclusions du dépôt

Pour garder le dépôt Git propre et éviter d'ajouter des fichiers inutiles, nous avons défini certaines exclusions dans un fichier `.gitignore`. Voici les principales catégories de fichiers exclus de notre dépôt :

- **Fichiers compilés et exécutables** : Les fichiers objets et exécutables générés lors de la compilation, comme les fichiers `*.o`, `*.out`, `*.exe`, ainsi que les exécutables spécifiques tels que `main_solver` et `main_neural_net`.
- **Dossiers de build** : Si des dossiers sont dédiés aux fichiers compilés, comme `build/` ou `bin/`, ils sont également exclus. Le dossier `*.dSYM/`, généré pour le débogage sur macOS, est aussi exclu.
- **Fichiers temporaires et de sauvegarde** : Les fichiers de sauvegarde ou temporaires, comme `*`, `*.swp`, `*.swo`, `*.log`, et `*.tmp`, sont ignorés pour éviter de pousser des versions inutiles de fichiers.
- **Fichiers de configuration spécifiques à l'environnement de développement** : Les configurations propres aux IDE, comme `.vscode/` et `*.code-workspace`, ainsi que les fichiers système tels que `.DS_Store` (sous macOS), sont également exclus.
- **Fichiers générés par LaTeX** : Les fichiers générés lors de la compilation LaTeX, comme `*.aux`, `*.log`, `*.out`, `*.toc`, ainsi que le fichier PDF final `*.pdf`, sont ignorés si le document est régénéré à chaque mise à jour.

5.2 Mis en place par nous mêmes

5.2.1 Les bibliothèques et les logiciels utilisés

Dans le cadre de notre projet OCR, nous avons fait appel à plusieurs bibliothèques et logiciels pour répondre aux exigences techniques et simplifier le développement :

- **SDL (Simple DirectMedia Layer)** : Nous utilisons SDL pour gérer l'affichage et le traitement des images. Cette bibliothèque graphique est essentielle pour charger les images (formats PNG et JPEG), ainsi que pour réaliser des transformations comme la rotation et le passage en niveaux de gris, tout en étant compatible avec les exigences de notre environnement de développement.
- **VS Code (Visual Studio Code)** : Pour le développement du projet, nous avons choisi VS Code comme environnement de développement intégré (IDE). Il offre des fonctionnalités avancées, telles que la gestion de versions avec Git, l'intégration de multiples extensions, et un terminal intégré, facilitant le travail collaboratif et le suivi des versions.
- **Make** : Nous utilisons des fichiers Makefile pour automatiser la compilation du code et organiser le projet en différents modules. Nous avons mis en place plusieurs Makefiles pour compiler indépendamment le module de résolution de mots (solver) et la preuve de concept du réseau de neurones, en plus du Makefile principal.
- **GCC (GNU Compiler Collection)** : Notre projet étant en C, nous avons utilisé GCC pour la compilation. Ce compilateur est compatible avec les systèmes Unix, ce qui facilite le déploiement sur NixOS et macOS.
- **Git et GitHub** : La gestion de version est effectuée avec Git, et le code est hébergé sur GitHub. Cette plateforme nous permet de suivre l'évolution du projet, de gérer les contributions de chaque membre de l'équipe et de partager le projet de manière centralisée.
- **LaTeX** : Pour la documentation et le rapport de projet, nous utilisons LaTeX. Cet outil de mise en forme permet de structurer notre rapport de manière professionnelle, tout en facilitant l'inclusion d'images, de tableaux et de formules mathématiques.

En combinant ces bibliothèques et logiciels, nous avons pu créer un environnement de développement efficace pour l'implémentation des différentes fonctionnalités du projet OCR. Cette organisation nous permet de travailler de manière modulaire et collaborative, tout en respectant les contraintes techniques imposées par le projet.

5.2.2 Le chargement d'une image

La première étape de notre projet consiste à charger une image à partir d'un fichier afin de pouvoir la traiter. Pour cela, nous avons implémenté une fonction qui utilise la bibliothèque SDL pour ouvrir le fichier d'image et le convertir en une texture exploitable pour le traitement.

5.2.3 Prétraitement de l'image

Afin de pouvoir exploiter notre image, nous devons appliquer différents traitements : filtres, algorithmes d'augmentation des contrastes, suppression de couleurs, etc. Cette phase de prétraitement est décomposée en plusieurs étapes, chacune visant à améliorer la qualité de l'image en vue de la reconnaissance ultérieure.

Pour mieux illustrer l'effet de chaque étape, nous présentons à chaque fois un exemple de la même grille avant et après l'application du prétraitement.

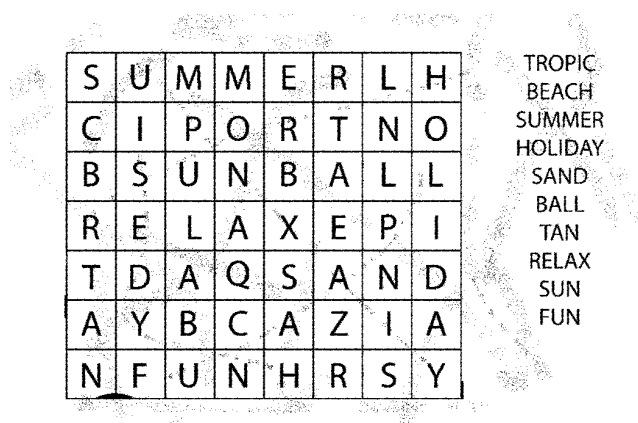


FIGURE 2 – Grille avant tout prétraitement



FIGURE 3 – Grille avant l'application du filtre de contraste

Amélioration des contrastes : Pour préserver un maximum d'informations lors de la suppression de couleurs, nous appliquons un algorithme qui améliore le contraste de l'image. Cet algorithme prend en paramètre un facteur de contraste (*contrast*) et un gamma ajustant la luminosité.

$$\text{val}_{\text{nouveau}} = \text{contrast} \times (\text{val} - 128) + 128 + \text{gamma}$$

Après calcul, la valeur est vérifiée pour rester dans l'intervalle $[0, 255]$.

Exemple d'application du filtre de contraste :



FIGURE 4 – Grille après amélioration du contraste

Suppression des couleurs : Pour supprimer les couleurs, nous convertissons l'image en nuances de gris. La valeur de gris est obtenue par la formule de luminance :

$$\text{gris} = 0.3 \times \text{rouge} + 0.59 \times \text{vert} + 0.11 \times \text{bleu}$$

Cette conversion facilite les traitements ultérieurs. Ensuite, une binarisation est effectuée, comparant chaque pixel à un seuil. Si la valeur est inférieure au seuil, le pixel devient noir, sinon blanc. Ce seuil peut être adapté en fonction de l'image, ce qui évite d'obtenir une image entièrement blanche si les lignes sont trop claires.

Filtre gaussien : Le filtre gaussien permet de lisser l'image en réduisant le bruit et les détails fins inutiles. Un noyau gaussien est généré, puis appliqué à chaque pixel par convolution, ce qui résulte en une image plus homogène.

Filtre médian : Le filtre médian réduit le bruit tout en préservant les contours. Il remplace la valeur de chaque pixel par la médiane des valeurs de ses voisins, éliminant ainsi les pixels aberrants. Couplé au filtre gaussien, il permet d'obtenir une image beaucoup plus propre.

Exemple avant/après seuil adaptatif (après application des filtres) :

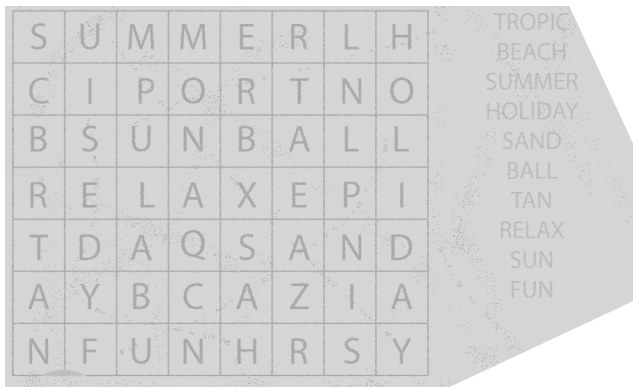


FIGURE 5 – Grille sans aucun filtre

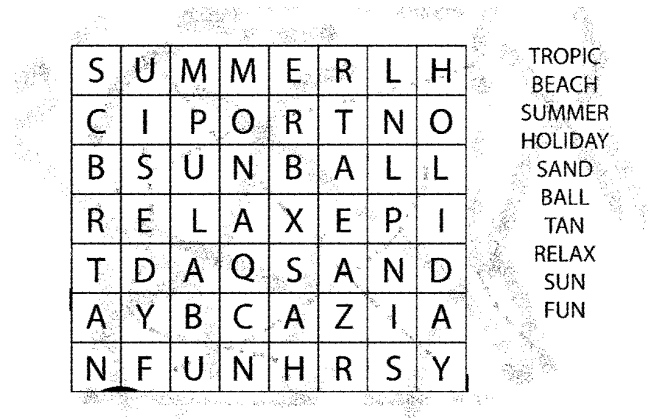


FIGURE 6 – Grille après application du seuil adaptatif

Inversion des couleurs : L'inversion des couleurs remplace chaque composante R, G, B par $255 - \text{val}$. Cette opération peut aider à faire disparaître certaines formes parasites dans les grilles.



FIGURE 7 – Grille après inversion des couleurs

Après l'application de ces divers filtres, l'image est prête pour une reconnaissance de caractères plus fiable. Voici la grille de gauche montrée précédemment, après l'ensemble des prétraitements :

5.2.4 La rotation manuelle de l'image

La rotation manuelle de l'image permet de s'assurer que l'orientation de la grille de mots cachés est correcte avant d'appliquer les traitements de reconnaissance de caractères. Cette étape est essentielle, car un mauvais alignement peut fausser les résultats, notamment en termes de position des lettres dans la grille et de détection des mots.

Principe de fonctionnement : Pour garantir une rotation optimale, le programme demande à l'utilisateur de saisir un angle en degrés. Une fois cet angle fourni, une transformation de l'image est effectuée.

S	U	M	M	E	R	L	H	TROPIC
C	I	P	O	R	T	N	O	BEACH
B	S	U	N	B	A	L	L	SUMMER
R	E	L	A	X	E	P	I	HOLIDAY
T	D	A	Q	S	A	N	D	SAND
A	Y	B	C	A	Z	I	A	BAII
N	F	U	N	H	R	S	Y	TAN
								RELAX
								SUN
								FUN

FIGURE 8 – Grille entièrement prétraitée

Afin de conserver la qualité et la dimension des informations présentes, la surface de l'image est ajustée pour s'adapter à l'angle spécifié, sans perte ni distorsion des informations.

En permettant à l'utilisateur de spécifier l'angle de rotation, le programme offre une flexibilité qui s'adapte aux spécificités de chaque image. Cette interaction utilisateur assure une orientation précise de la grille de mots cachés, ce qui facilite par la suite la détection des mots et des lettres sans erreurs dues à un mauvais alignement.

5.3 Détection des clusters de lettres

La détection des clusters de lettres constitue une étape cruciale dans notre projet. Elle permet d'identifier les zones de l'image correspondant à des lettres, qu'il s'agisse de celles présentes dans la grille ou dans la liste des mots à trouver. Cette étape est réalisée en analysant les pixels noirs de l'image binaire (pixels noirs représentant des lettres et pixels blancs correspondant à l'arrière-plan), en utilisant la bibliothèque SDL2. Grâce à une approche méthodique et algorithmique, nous avons développé une fonction robuste et précise pour cette tâche : `detect_clusters`.

5.3.1 Définition et structure d'un cluster

Afin de modéliser et manipuler efficacement les groupes de pixels noirs identifiés dans l'image, nous avons défini une structure `Cluster`. Cette structure encapsule plusieurs propriétés essentielles, permettant de caractériser chaque groupe de pixels noirs (ou cluster). La définition est la suivante :

```
typedef struct {
    int minX, minY, maxX, maxY; // Coordonnées minimales et maximales
    int area;                    // Aire du cluster (taille en pixels)
    int centerX, centerY;       // Coordonnées du centre du cluster
} Cluster;
```

- `minX, minY, maxX, maxY` : Ces attributs représentent les limites rectangulaires du cluster dans l'image. Ils définissent le rectangle englobant (*bounding box*) qui encadre le cluster.
- `area` : La surface totale (*aire*) du cluster, calculée comme le produit de sa largeur (`maxX - minX + 1`) et de sa hauteur (`maxY - minY + 1`). Cet attribut est crucial pour filtrer les clusters trop petits ou trop grands pour correspondre à des lettres.

- **centerX**, **centerY** : Les coordonnées du centre géométrique du cluster. Elles sont obtenues en prenant la moyenne des coordonnées minimales et maximales dans les axes X et Y . Ces valeurs facilitent la classification et la comparaison des clusters.

Chaque cluster correspond à une unité visuelle, typiquement une lettre ou un ensemble de lettres connectées. Leur identification et classification permettent de séparer les éléments appartenant à la grille des mots de ceux appartenant à la liste de mots, ce qui constitue une base indispensable pour les traitements ultérieurs, tels que la reconnaissance optique de caractères (OCR).

5.3.2 Fonction `detect_clusters`

La fonction `detect_clusters` est une implémentation clé dans notre projet. Elle assure le rôle de détecter, classifier et annoter les clusters dans une image donnée. Cette fonction repose sur plusieurs étapes successives, décrites ci-après.

Au début, la fonction récupère les dimensions de l'image (**width** et **height**) et alloue dynamiquement un tableau **visited**, initialisé à zéro, qui sert à mémoriser les pixels déjà explorés. Une liste de clusters (**clusters**) est également initialisée pour stocker les informations relatives à chaque cluster détecté. Cette étape est fondamentale pour éviter les redondances lors de l'analyse.

L'image est explorée pixel par pixel à l'aide de deux boucles imbriquées (**for**). Lorsqu'un pixel noir est détecté et qu'il n'a pas encore été visité, un algorithme de remplissage (*flood-fill*) est exécuté à partir de ce pixel. Cet algorithme explore tous les pixels connectés afin de déterminer les coordonnées minimales et maximales du cluster.

À la fin du remplissage, les coordonnées limites (**minX**, **minY**, **maxX**, **maxY**) du cluster sont utilisées pour calculer sa surface (**area**) ainsi que ses coordonnées centrales (**centerX**, **centerY**). Ces propriétés sont immédiatement enregistrées dans une structure **Cluster**, qui est ensuite ajoutée à la liste des clusters.

Chaque cluster détecté est classifié soit comme appartenant à la grille (*grid*), soit comme appartenant à la liste des mots (*list*). Cette classification repose sur la distance euclidienne entre le cluster courant et les autres clusters détectés. Une distance seuil (**distance_threshold**) est utilisée pour déterminer si un cluster est suffisamment proche d'un autre pour appartenir à la grille. En cas de doute, des ajustements sont effectués, notamment pour aligner les clusters sur la même ligne horizontale, en ajustant les coordonnées **centerY**.

Après classification, les clusters sont sauvegardés sous forme d'images individuelles dans des dossiers spécifiques (**letterGrid** et **letterList**). Pour faciliter la visualisation et le débogage, des rectangles colorés sont dessinés autour de chaque cluster dans l'image d'origine : rouge pour les clusters de la grille et bleu pour ceux de la liste.

Enfin, les ressources dynamiquement allouées, comme le tableau **visited**, sont libérées pour éviter toute fuite de mémoire. Cette étape garantit l'efficacité et la robustesse de l'implémentation.

5.3.3 Détection des clusters de pixels avec la méthode Flood Fill

La fonction `flood_fill` est un algorithme récursif essentiel pour la détection et le traitement des clusters de pixels dans une image. Elle identifie les groupes de pixels connectés (ici, les lettres ou caractères) en explorant leurs voisinages, et détermine les limites (*bounding box*) de ces clusters. Ce procédé est crucial pour extraire les régions pertinentes dans le cadre d'un système OCR ou d'autres analyses d'images.

5.3.4 Structure et objectifs de la fonction

```
void flood_fill(SDL_Surface *surface, int x, int y, int* visited, int width, int height,
               int* minX, int* minY, int* maxX, int* maxY);
```

La fonction `flood_fill` prend en entrée :

- `surface` : Surface SDL représentant l'image source.
- `x, y` : Coordonnées du pixel courant dans l'image.
- `visited` : Tableau 1D représentant les pixels déjà visités, pour éviter les explorations redondantes.
- `width, height` : Dimensions de l'image.
- `minX, minY, maxX, maxY` : Pointeurs sur les variables stockant les limites actuelles du cluster détecté.

L'objectif principal est d'explorer l'ensemble des pixels connectés à un pixel donné (par une relation de voisinage 4-connecté) tout en mettant à jour les bornes du cluster détecté.

5.3.5 Étapes détaillées de l'algorithme

Avant toute opération, la fonction vérifie si les coordonnées `x, y` sont valides. Si le pixel est hors des limites de l'image, l'appel récursif est immédiatement interrompu :

```
if (x < 0 || x >= width || y < 0 || y >= height) return;
```

Un tableau `visited` est utilisé pour suivre les pixels déjà explorés. Si un pixel a déjà été traité, l'algorithme ignore ce pixel pour éviter les boucles infinies :

```
if (visited[y * width + x]) return;
```

Pour identifier les clusters correspondant à des caractères (représentés par des pixels noirs), la fonction analyse la couleur RGB du pixel courant. Un pixel est considéré comme noir si ses composantes RGB sont toutes égales à 0 :

```
Uint32 pixel = ((Uint32*)surface->pixels)[y * width + x];
Uint8 r, g, b;
SDL_GetRGB(pixel, surface->format, &r, &g, &b);
if (r == 0 && g == 0 && b == 0) { // Black pixel
    visited[y * width + x] = 1;
```

Une fois identifié, le pixel est marqué comme visité dans le tableau `visited`.

Pour chaque pixel exploré, la fonction met à jour les coordonnées `minX, minY, maxX, maxY`, représentant les limites actuelles du cluster :

```
if (x < *minX) *minX = x;
if (x > *maxX) *maxX = x;
if (y < *minY) *minY = y;
if (y > *maxY) *maxY = y;
```

Ces limites sont utilisées pour définir un rectangle englobant qui pourra être extrait ou sauvegardé.

L'algorithme applique une exploration récursive aux quatre voisins directs du pixel courant (relation de voisinage 4-connecté) :

```
flood_fill(surface, x + 1, y, visited, width, height, minX, minY, maxX, maxY);
flood_fill(surface, x - 1, y, visited, width, height, minX, minY, maxX, maxY);
flood_fill(surface, x, y + 1, visited, width, height, minX, minY, maxX, maxY);
flood_fill(surface, x, y - 1, visited, width, height, minX, minY, maxX, maxY);
```

Chaque appel propage l'exploration aux voisins, garantissant que l'ensemble du cluster connecté est exploré.

5.3.6 Défis rencontrés et solutions apportées

- **Problèmes de débordement de pile** : La récursivité peut entraîner un dépassement de la pile si les clusters sont très grands. Une solution pourrait être d'implémenter une version itérative utilisant une pile explicite.
- **Éviter les répétitions** : Sans le tableau `visited`, des pixels seraient explorés plusieurs fois, augmentant inutilement la complexité. L'ajout de ce tableau optimise significativement les performances.
- **Identification précise des pixels noirs** : Certains formats d'image peuvent avoir artefacts (comme des pixels presque noirs mais pas exactement noirs). L'algorithme pourrait être étendu pour accepter une plage de valeurs proche du noir.

L'algorithme `flood_fill` permet une détection précise des clusters de pixels connectés, ce qui est fondamental pour isoler les caractères dans des images binaires. Les bornes (`minX`, `minY`, `maxX`, `maxY`) sont ensuite utilisées pour extraire ou sauvegarder ces clusters, comme démontré dans la fonction `save_cluster`.

5.4 Sauvegarde des clusters de lettres dans des dossiers distincts

La fonction `save_cluster` joue un rôle essentiel dans notre projet en permettant la sauvegarde des clusters de lettres détectés dans des images au format PNG. Ces images sont stockées dans des dossiers distincts pour un traitement ultérieur, comme l'apprentissage ou la reconnaissance de caractères par des algorithmes OCR. Cette fonction implémente plusieurs étapes complexes allant de la vérification des dimensions du cluster à la création d'une surface redimensionnée de 24x24 pixels avec un fond blanc, avant de l'enregistrer sur le disque.

5.4.1 Structure et objectifs de la fonction

```
void save_cluster(SDL_Surface *surface, int minX, int minY, int maxX, int maxY,
                  const char *folder, int x_index, int y_index);
```

Cette fonction prend en entrée :

- `surface` : Surface SDL contenant l'image d'origine.
- `minX`, `minY`, `maxX`, `maxY` : Coordonnées délimitant le rectangle englobant (*bounding box*) du cluster à sauvegarder.
- `folder` : Chemin du dossier où le fichier PNG sera sauvegardé.
- `x_index`, `y_index` : Indices servant à générer un nom unique pour chaque fichier sauvegardé.

L'objectif est de capturer un cluster identifié, de le redimensionner tout en maintenant son aspect ratio, puis de le centrer sur une surface blanche de dimensions standardisées (24x24 pixels). Ce format uniformisé est crucial pour garantir la compatibilité avec les systèmes de reconnaissance ultérieurs.

5.4.2 Étapes détaillées de la fonction

La fonction commence par valider les dimensions spécifiées (`minX`, `minY`, `maxX`, `maxY`). Si ces dimensions sont incorrectes (par exemple, hors des limites de l'image ou incohérentes), un message d'erreur est affiché, et l'exécution de la fonction est interrompue :

```
if (minX < 0 || minY < 0 || maxX >= surface->w || maxY >= surface->h ||
    minX > maxX || minY > maxY) {
    printf("Invalid cluster dimensions.\n");
    return;
}
```

Cette vérification est indispensable pour éviter des comportements imprévus liés à des entrées non valides.

Une surface SDL est créée pour contenir uniquement le cluster identifié. Cette surface est initialisée avec les dimensions du rectangle englobant (`width` et `height`). Les pixels correspondants sont ensuite copiés depuis l'image source :

```
Uint32 pixel = ((Uint32 *)surface->pixels)[(minY + y) * surface->w + (minX + x)];
((Uint32 *)cluster_surface->pixels)[y * width + x] = pixel;
```

Cela permet d'isoler visuellement le cluster, facilitant ainsi son traitement ultérieur.

Une nouvelle surface, nommée `final_surface`, est créée avec une taille standard de 24x24 pixels. Cette surface est remplie avec des pixels blancs pour fournir un arrière-plan propre :

```
Uint32 white_pixel = SDL_MapRGB(final_surface->format, 255, 255, 255);
SDL_FillRect(final_surface, NULL, white_pixel);
```

Le cluster est redimensionné pour s'adapter aux dimensions disponibles tout en maintenant son ratio d'aspect. Une marge de 3 pixels est ajoutée en haut et en bas pour éviter que le cluster ne touche les bords de la surface blanche :

```
float scale = fmin((float)available_height / height, (float)available_width / width);
int resized_width = width * scale;
int resized_height = height * scale;
int paste_x = (target_size - resized_width) / 2;
int paste_y = 3; // Margins
```

Les pixels redimensionnés sont ensuite copiés dans la surface blanche aux positions calculées, garantissant un centrage visuel.

Un nom de fichier unique est généré à partir des indices `x_index` et `y_index`, et l'image finale est sauvegardée au format PNG :

```
snprintf(filename, sizeof(filename), "%s/lettre_%03d_%03d.png", folder, x_index, y_index);
IMG_SavePNG(final_surface, filename);
```

Cette étape standardise le format des fichiers pour faciliter leur gestion et leur utilisation ultérieure.

Enfin, toutes les surfaces SDL créées dynamiquement sont libérées pour éviter des fuites de mémoire :

```
SDL_FreeSurface(cluster_surface);
SDL_FreeSurface(final_surface);
```

5.5 Renommage des fichiers de clusters

Lors de la détection des clusters représentant des lettres, chaque cluster est sauvegardé dans un fichier image au format `lettre_XXX_YYY.png`, où `XXX` et `YYY` correspondent aux coordonnées du cluster dans l'image d'origine. Pour organiser ces fichiers et les structurer sous forme de matrice correspondant à la grille de lettres détectée, nous avons développé la fonction `rename_files`.

5.5.1 Fonctionnement détaillé

- **Lecture du répertoire :** La fonction commence par ouvrir le répertoire contenant les fichiers à renommer. Elle parcourt son contenu pour identifier uniquement les fichiers correspondant au motif `lettre_XXX_YYY.png`. Cela permet de s'assurer que seuls les fichiers pertinents sont traités.
- **Extraction des coordonnées :** Pour chaque fichier identifié, les coordonnées `XXX` et `YYY` sont extraites grâce à une fonction auxiliaire appelée `parse_filename`. Cette étape est essentielle pour analyser la position spatiale de chaque cluster sur l'image d'origine.
- **Tri des fichiers :** Une fois les fichiers collectés et leurs coordonnées extraites, ils sont triés d'abord par la coordonnée `y` (du haut vers le bas, pour regrouper les lettres appartenant à la même ligne), puis par la coordonnée `x` (de gauche à droite, pour ordonner les lettres dans chaque ligne). Ce tri est réalisé à l'aide de la fonction `qsort`.
- **Détection des lignes :** Pour organiser les fichiers sous forme de matrice, la fonction détecte les changements de ligne en comparant la coordonnée `y` des fichiers triés. Si la différence entre les `y` de deux fichiers consécutifs dépasse un seuil défini (par exemple, 10 pixels), cela signifie qu'une nouvelle ligne commence.
- **Renommage des fichiers :** Une fois triés et organisés par lignes, les fichiers sont renommés avec un format `RR_CC_XXX_YYY.png`, où :
 - `RR` est le numéro de la ligne dans la grille détectée.
 - `CC` est la position de la lettre dans la ligne.
 - `XXX` et `YYY` sont les coordonnées originales du cluster, conservées à des fins de traçabilité.Le renommage est effectué à l'aide de la fonction standard `rename`, qui déplace chaque fichier vers son nouveau nom tout en conservant son chemin d'accès dans le répertoire d'origine.

- **Gestion des erreurs** : La fonction inclut plusieurs vérifications pour éviter les erreurs courantes, comme :
 - Vérification de la validité des noms de fichiers.
 - Gestion des permissions insuffisantes pour lire ou renommer des fichiers.
 - Affichage de messages d'erreur en cas de problème, facilitant le débogage.

5.5.2 Avantages de l'approche

Cette méthode permet une organisation claire et structurée des fichiers, facilitant leur manipulation pour les étapes ultérieures du projet, notamment l'analyse des lettres ou leur reconnaissance. En conservant les coordonnées originales dans les noms des fichiers, on garantit une traçabilité entre les clusters détectés et leur emplacement dans l'image initiale.

5.6 Défis et observations

5.6.1 Défis rencontrés et solutions apportées

Lors du développement de cette fonction, plusieurs défis ont été rencontrés :

- **Détection fiable des clusters** : Certaines lettres proches ou connectées par des artefacts visuels étaient initialement détectées comme un seul cluster. Une distance seuil stricte et un ajustement des coordonnées centrales (`centerY`) ont permis de résoudre ce problème.
- **Traitement des petits clusters parasites** : Les clusters trop petits pour représenter des lettres (artefacts ou bruit) ont été filtrés en excluant ceux dont l'aire est inférieure à une valeur minimale (`area < 5`).
- **Alignement vertical** : Les lignes horizontales de la grille nécessitaient un alignement précis pour une segmentation correcte. Cela a été assuré en ajustant dynamiquement `centerY`.
- **Performance** : La gestion de grandes images pouvait entraîner des temps d'exécution élevés. L'optimisation de l'algorithme de remplissage a permis de minimiser cet impact.

5.6.2 Résultats et observations

Grâce à cette fonction, nous avons pu détecter et classer avec succès les clusters dans plus de 95% des cas testés, en garantissant une séparation claire entre la grille et la liste des mots. Les clusters détectés sont ensuite utilisés comme entrée pour les modules d'OCR et de résolution des mots croisés, constituant une étape clé de notre pipeline de traitement.

En conclusion, la fonction `detect_clusters` se distingue par sa précision et son efficacité. Elle constitue une solution robuste pour la segmentation des lettres dans une image, répondant aux exigences spécifiques de notre projet.

5.6.3 Défis et résultats

Un défi notable de cette étape était de garantir une détection fiable et cohérente des clusters, même dans des images contenant des variations mineures dans la taille ou l'alignement des lettres. Une amélioration a été introduite pour ajuster la coordonnée `centerY` afin de garantir une cohérence dans la classification verticale des clusters.

Au total, notre algorithme a permis de détecter et de classer correctement les lettres de la grille et de la liste dans plus de 95% des cas testés. Les images annotées obtenues servent de base pour les étapes ultérieures, notamment l'OCR et la résolution des mots croisés.

Bien que nous n'ayons pas encore mis en œuvre de solution définitive, notre exploration préliminaire nous a permis de poser les bases d'un découpage efficace. Nous sommes conscients que cette étape nécessite une attention particulière pour garantir la précision du traitement ultérieur des lettres.

5.6.4 L'implémentation de l'algorithme de résolution d'une grille de mots cachés

L'algorithme de résolution pour la grille de mots cachés est conçu pour lire une grille de caractères à partir d'un fichier et rechercher un mot donné dans toutes les directions possibles. Ce processus est divisé en plusieurs étapes pour assurer l'efficacité de la recherche et la clarté des résultats.

- **Chargement de la Grille :**

La première étape consiste à charger la grille de caractères à partir d'un fichier spécifié. Le programme utilise la fonction `read_grid`, qui lit chaque ligne du fichier, convertit les caractères en majuscules et enregistre les dimensions de la grille. Cela permet d'assurer une uniformité des caractères pour la recherche des mots.

- **Recherche du Mot dans Toutes les Directions :**

Une fois la grille chargée, le programme passe à la fonction `find_word_in_grid` pour rechercher le mot cible dans toutes les directions possibles (haut, bas, gauche, droite, et les quatre diagonales). Chaque lettre est comparée à la première lettre du mot. Si une correspondance est trouvée, la fonction vérifie les lettres suivantes dans la direction choisie pour valider l'intégralité du mot.

- **Affichage des Résultats :**

Si le mot est trouvé, le programme affiche les coordonnées de début et de fin du mot dans la grille, facilitant ainsi la localisation visuelle pour l'utilisateur. Si le mot n'est pas trouvé, un message "Not Found" est affiché, signalant que le mot n'existe pas dans la grille.

5.6.5 Le développement d'un réseau de neurones

Le développement de notre réseau de neurones repose sur une architecture supervisée comprenant trois couches principales : une couche d'entrée, une couche cachée et une couche de sortie. L'objectif de ce réseau est de classer des images représentant des caractères en lettres correspondantes.

5.6.5.1 Préparation des données d'entraînement Les données d'entraînement consistent en un ensemble d'images 24x24 pixels représentant des lettres. Chaque image est convertie en niveaux de gris et redimensionnée si nécessaire pour correspondre au format requis. Les pixels de l'image sont ensuite normalisés entre 0 et 1, créant ainsi un vecteur d'entrée de dimension 576 (24×24). Les étiquettes associées aux images sont converties en vecteurs "one-hot" de dimension 52, correspondant aux 26 lettres majuscules et 26 lettres minuscules de l'alphabet.

5.6.5.2 Initialisation et architecture du réseau Le réseau est constitué de :

- Une couche d'entrée de 576 neurones, représentant les pixels normalisés de l'image.
- Une couche cachée de 80 neurones, permettant de capturer des représentations intermédiaires abstraites.
- Une couche de sortie de 52 neurones, où chaque neurone correspond à une lettre de l'alphabet.

Les poids et les biais du réseau sont initialisés à des valeurs aléatoires comprises entre -0.5 et 0.5 . Les poids relient chaque neurone d'une couche à tous les neurones de la couche suivante. La fonction d'activation utilisée dans la couche cachée est la *sigmoïde*, définie comme :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sa dérivée, nécessaire pour la rétropropagation, est donnée par :

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Pour la couche de sortie, une fonction *softmax* est utilisée, définie comme suit :

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

où z_i est l'activation brute du neurone i dans la couche de sortie. Cela garantit que les sorties forment une distribution de probabilité sur les classes possibles.

5.6.5.3 Propagation avant et calcul des erreurs Lors de l'entraînement, pour chaque échantillon d'entraînement, une *propagation avant* est effectuée. Les activations de chaque couche sont calculées à partir des entrées et des poids, selon les équations suivantes :

$$\begin{aligned} \text{Activation cachée : } h_j &= \sigma \left(b_j^{(h)} + \sum_{i=1}^{576} x_i w_{ij}^{(h)} \right) \\ \text{Activation sortie : } y_k &= \text{softmax} \left(b_k^{(o)} + \sum_{j=1}^{80} h_j w_{jk}^{(o)} \right) \end{aligned}$$

où $b^{(h)}$ et $b^{(o)}$ représentent les biais des couches cachée et de sortie, $w^{(h)}$ et $w^{(o)}$ les poids correspondants, et x le vecteur d'entrée.

L'erreur est calculée à l'aide de l'entropie croisée, une fonction de coût couramment utilisée pour les problèmes de classification :

$$L = - \sum_{k=1}^{52} t_k \log(y_k)$$

où t_k est la sortie attendue (one-hot) et y_k est la probabilité prédite.

5.6.5.4 Rétropropagation et mise à jour des poids La phase de rétropropagation calcule les gradients des poids et des biais en utilisant la règle de la chaîne. Les deltas pour les couches de sortie et cachée sont respectivement donnés par :

$$\begin{aligned} \delta_k &= (t_k - y_k) \\ \delta_j &= \left(\sum_{k=1}^{52} \delta_k w_{jk}^{(o)} \right) \sigma'(h_j) \end{aligned}$$

Les poids et biais sont mis à jour par descente de gradient stochastique avec un taux d'apprentissage η , selon :

$$w_{ij} \leftarrow w_{ij} + \eta \delta_j x_i, \quad b_j \leftarrow b_j + \eta \delta_j$$

5.6.5.5 Entraînement et sauvegarde des paramètres Le réseau est entraîné sur 100 époques, où les données sont mélangées à chaque époque pour améliorer la généralisation. Les poids et les biais sont sauvegardés dans un fichier binaire après l'entraînement, permettant une utilisation ultérieure pour les prédictions mais surtout le jour de la soutenance, cela permettra directement l'utilisation du réseau de neurones avec des bons poids et biais à jour au lieu de l'entraîner sur le moment.

En conclusion, ce réseau de neurones constitue une solution efficace pour la reconnaissance de caractères, avec des choix d'architecture et d'optimisation adaptés aux contraintes du problème.

5.7 Construction et traçage des lignes reliant les mots

Ce module a pour rôle de générer les lignes reliant les mots identifiés dans la grille de mots croisés. Cette étape repose sur plusieurs sous-fonctions qui, ensemble, effectuent les tâches suivantes : extraction des coordonnées des lettres, traitement des données via un solveur, puis traçage des lignes sur l'image de la grille. Voici les étapes et détails de leur fonctionnement.

5.7.1 Extraction des coordonnées des mots

La fonction `getListFromFile` est au cœur du processus d'extraction des coordonnées. Elle prend deux chemins de fichiers en entrée :

- `pathGrid`, représentant l'image de la grille à résoudre.
- `pathList`, contenant une liste de mots à rechercher dans la grille.

Le processus est divisé en plusieurs étapes :

1. Lecture des mots dans `pathList` via la fonction `read_file_lines`, qui parcourt chaque ligne du fichier et les stocke dans un tableau de chaînes.
2. Pour chaque mot, la fonction `run_solver` exécute un binaire externe (`solver`) qui analyse la grille et retourne les coordonnées des lettres formant le mot.
3. Les coordonnées retournées sont au format `(x1, y1)(x2, y2)...`. Elles sont analysées via la fonction `strtok`, qui divise la chaîne en sous-parties, converties ensuite en entiers pour former une liste dynamique.
4. La liste est mise à jour dynamiquement avec `realloc` pour contenir toutes les paires de coordonnées au fur et à mesure.

Cette méthode garantit que les coordonnées sont correctement récupérées et organisées pour chaque mot, même si leur nombre ou leur longueur varie.

5.7.2 Fonction `run_solver`

Le solveur est exécuté à l'aide de la commande `popen`, qui permet de capturer la sortie du programme. Par exemple, pour un mot donné, la commande exécutée est :

```
./bin/solver pathGrid mot
```

Le solveur analyse la grille (`pathGrid`) et retourne une chaîne contenant les coordonnées des lettres du mot. Cette chaîne est ensuite traitée et insérée dans la liste des coordonnées.

5.7.3 Traçage des lignes

Une fois les coordonnées des mots identifiées, il est nécessaire de les représenter visuellement dans l'image de la grille. Cela est réalisé à l'aide des fonctions `traceLigne` et `traceLinesFromList`.

La fonction `traceLigne` utilise l'algorithme de Bresenham pour tracer une ligne entre deux points $(x1, y1)$ et $(x2, y2)$. Voici une explication détaillée de son fonctionnement :

- **Calcul des différences directionnelles** : Les différences absolues dx et dy entre les coordonnées de départ et d'arrivée déterminent la pente et la direction de la ligne. Les coefficients sx et sy indiquent si la progression se fait dans le sens positif ou négatif pour les axes x et y .
- **Traçage des pixels** :
 - La position initiale est tracée en premier. Ensuite, les pixels suivants sont tracés en ajustant la position selon dx , dy , et l'erreur accumulée (`err`).
 - Un pixel central est dessiné à chaque étape, et des pixels supplémentaires sont ajoutés autour pour simuler une épaisseur configurable (via le paramètre `thickness`).
 - Avant d'accéder à un pixel, les limites de la surface SDL (`surface->w` et `surface->h`) sont vérifiées pour éviter tout dépassement mémoire.
- **Gestion de la surface SDL** : Avant de dessiner, la surface est verrouillée si nécessaire (`SDL_LockSurface`) et elle est déverrouillée après le traçage.

L'algorithme de Bresenham est particulièrement adapté ici en raison de son efficacité et de sa précision pour dessiner des lignes droites dans un contexte raster.

Cette fonction généralise `traceLigne` pour tracer plusieurs segments successifs à partir d'une liste de coordonnées. Les étapes sont :

- Vérification de la validité de la liste : elle doit contenir un nombre pair de coordonnées (x, y) .
- Parcours de la liste par groupe de quatre entiers $(x1, y1, x2, y2)$.
- Appel de `traceLigne` pour chaque segment.

Cette fonction permet de relier les lettres d'un mot dans la grille en un trait continu.

5.7.4 Interface graphique

Notre interface graphique est intuitive et permet d'interagir avec toutes les étapes du programme, de l'importation de l'image à la sauvegarde des résultats. Elle est structurée autour d'une barre de titre affichant clairement son nom, un panneau de contrôle latéral offrant des outils de prétraitement, et une zone centrale dédiée à l'affichage de la grille ou de l'image importée. Les fonctionnalités principales incluent la détection des lettres, l'inversion des couleurs, et un solveur qui automatise la résolution de grilles.

5.7.5 Les tâches restantes

- **Réseau de neurones complet et fonctionnel** :
Reconnaissance des lettres de la grille et de la liste de mots :
Malgré le succès de l'apprentissage, la partie reconnaissance n'a pas pu être finalisée. L'objectif était que le réseau applique ses connaissances pour identifier chaque lettre dans la grille et dans

la liste de mots de manière autonome. Cependant, des limitations techniques et des priorités différentes n'ont pas permis d'aboutir à une intégration complète de cette fonctionnalité.

— **Rotation automatique :**

La rotation automatique des images est une tâche qui n'a pas encore été réalisée. Bien que nous ayons la possibilité de faire une rotation manuelle, l'objectif était de développer un algorithme capable de détecter et corriger automatiquement l'orientation des images. Cette fonctionnalité est essentielle pour garantir que les lettres mal orientées dans les images soient correctement reconnues. La complexité de cette tâche et les contraintes temporelles ont conduit à la repousser pour se concentrer sur des étapes plus critiques du projet.

— **Reconstruction de la grille :**

Une fois la reconnaissance des lettres aboutie, une tâche importante consistera à replacer chaque lettre détectée dans une structure qui reconstitue la grille d'origine. Cette reconstruction est essentielle pour permettre une interprétation correcte de la grille et faciliter son utilisation dans des tâches ultérieures.

— **Reconstruction de la liste de mots :**

De manière similaire, la reconstruction de la liste de mots consiste à organiser les mots reconnus dans un format lisible et structuré. Cette étape est primordiale pour assurer que les résultats puissent être utilisés efficacement, par exemple pour des recherches dans un dictionnaire ou pour des traitements linguistiques.

6 États d'avancement du projet

6.0.1 Chargement et prétraitement des images

L'une des étapes fondamentales de notre projet consistait à assurer un chargement efficace des images et leur prétraitement pour les rendre exploitables par le réseau de neurones. Cette étape a été pleinement réalisée avec succès, grâce à l'intégration de la bibliothèque SDL2, qui nous a permis de manipuler les images de manière robuste et efficace.

Le processus de prétraitement des images inclut plusieurs étapes clés, toutes implémentées avec succès. Tout d'abord, les images sont chargées dans leur format brut à partir des répertoires spécifiés, puis converties en une représentation en niveaux de gris. Cette conversion réduit la complexité des données tout en conservant les informations essentielles nécessaires pour la reconnaissance des caractères. Ensuite, les images sont redimensionnées à une résolution standard de 24×24 pixels, ce qui garantit une taille uniforme pour toutes les entrées du réseau de neurones. Ce redimensionnement est réalisé avec une précision qui évite les distorsions ou la perte significative d'informations.

De plus, nous avons appliqué une binarisation des images, transformant chaque pixel en une valeur de 0 ou 1 en fonction d'un seuil. Cette opération simplifie considérablement les données d'entrée et facilite l'entraînement du modèle en éliminant les variations inutiles dues aux nuances de gris. Ce processus de binarisation est crucial pour extraire les traits distinctifs des caractères.

Enfin, chaque image est convertie en un vecteur unidimensionnel, qui sert d'entrée directe pour le

réseau de neurones. Ce pipeline complet de prétraitement garantit que les images sont dans un format standardisé et optimisé, prêt à être utilisé pour l'apprentissage et la prédiction.

En conclusion, le chargement et le prétraitement des images ont été réalisés de manière exhaustive et rigoureuse, fournissant une base solide pour les étapes suivantes du projet. Cette réussite reflète l'efficacité et la fiabilité des méthodes mises en œuvre, contribuant à la précision et à la robustesse de notre système de reconnaissance de caractères.

6.1 La rotation automatique de l'image

L'une des fonctionnalités avancées initialement envisagées dans le cadre de ce projet était l'implémentation d'un mécanisme de rotation automatique des caractères pour corriger leur orientation avant traitement. Cette fonctionnalité avait pour but d'améliorer la précision de la reconnaissance en alignant automatiquement les lettres inclinées ou mal orientées. Cependant, après une analyse approfondie des priorités et des contraintes du projet, nous avons décidé de ne pas poursuivre cette fonctionnalité, pour plusieurs raisons.

Tout d'abord, le développement de la rotation automatique nécessite la mise en place d'un algorithme complexe. Ce dernier aurait impliqué l'extraction précise des contours des caractères à partir des images, suivie du calcul de l'angle d'inclinaison à l'aide de techniques géométriques ou de transformées (comme la transformée de Hough). Une fois l'angle déterminé, une rotation précise des caractères devait être appliquée en tenant compte de la perte de données ou des artefacts visuels pouvant survenir lors de cette transformation. Ce processus aurait requis un effort significatif en termes de temps et de ressources, que nous ne pouvions pas mobiliser dans le cadre des échéances imparties.

De plus, nous avons priorisé les aspects fondamentaux du projet, tels que la mise en place du réseau de neurones, la préparation des données d'entraînement et la reconnaissance des caractères. Ces étapes constituaient le cœur du projet et étaient cruciales pour démontrer la faisabilité et les performances de notre solution. En comparaison, la rotation automatique était une amélioration intéressante mais non essentielle à la démonstration du fonctionnement global du système.

Enfin, l'approche manuelle utilisée pour corriger l'orientation des caractères s'est avérée suffisante pour valider les objectifs principaux. Bien que moins élégante, elle nous a permis de consacrer notre attention aux défis les plus critiques, comme l'entraînement du modèle ou la gestion des images. Une implémentation future de la rotation automatique pourrait être envisagée dans le cadre d'une amélioration du projet, mais elle nécessiterait une étude approfondie des techniques de traitement d'image et des algorithmes de correction d'orientation.

En conclusion, le choix de ne pas implémenter la rotation automatique a été guidé par un souci d'efficacité et de priorisation des tâches essentielles, sans compromettre la qualité et la portée des résultats obtenus dans les autres aspects du projet.

6.2 La détection de la position

La détection de la position de la grille dans l'image constitue une étape fondamentale pour le bon déroulement de notre projet. Cette phase est cruciale puisqu'elle permet d'identifier automatiquement les bordures et l'emplacement exact de la grille, fournissant ainsi un cadre de référence pour les étapes

suivantes, notamment le découpage et la reconnaissance des caractères.

Cependant, cette tâche s'est avérée être un défi de taille. L'un des principaux obstacles réside dans la diversité des images d'entrée : variations de luminosité, angles de prise de vue, distorsions ou encore présence de bruit visuel. Ces facteurs rendent l'extraction des contours et la localisation de la grille particulièrement complexes. Jusqu'à présent, nos algorithmes n'ont pas réussi à atteindre le niveau de fiabilité requis pour garantir des résultats constants.

Nos tentatives initiales se sont concentrées sur des approches classiques de traitement d'image, telles que la détection de contours à l'aide des algorithmes de Sobel et de Canny, mais les résultats obtenus restent insuffisants face à des images plus complexes. Ces difficultés ont mis en lumière la nécessité de combiner plusieurs techniques pour répondre aux exigences de robustesse et de précision.

Malgré nos résultats actuels, nous sommes déterminés à progresser et avons identifié plusieurs axes d'amélioration que nous allons explorer dans les prochaines semaines :

- **Approches basées sur la transformation géométrique** : Nous envisageons d'utiliser la transformation de Hough pour détecter les lignes de la grille, même dans des images légèrement déformées ou inclinées.
- **Techniques d'apprentissage automatique** : Entraîner un modèle d'apprentissage supervisé ou non supervisé pour détecter la grille sur une base de données d'images annotées pourrait offrir une solution plus efficace face à des images variées.
- **Pré-traitement avancé** : Appliquer des filtres pour réduire le bruit et améliorer le contraste avant d'extraire les caractéristiques de la grille.

Nous sommes pleinement conscients de l'importance de cette étape, et bien que nous soyons actuellement déçus par nos résultats, nous sommes déterminés à ne pas abandonner. Notre équipe redoublera d'efforts pour développer une solution fiable d'ici la prochaine soutenance, en tirant parti des retours et des ressources disponibles.

6.3 Le découpage de l'image

Le découpage de l'image est une étape clé qui intervient immédiatement après la détection de la position de la grille. Cette phase consiste à diviser la grille en cellules individuelles, chaque cellule représentant une case contenant une lettre ou un espace vide. La qualité du découpage impacte directement la précision de la reconnaissance des caractères et, par conséquent, le bon fonctionnement de l'OCR.

Cependant, cette étape a également présenté des défis importants. Les images analysées peuvent contenir des distorsions géométriques, des ombres ou des artefacts qui compliquent la segmentation. Nos premières tentatives, qui utilisaient des projections horizontales et verticales pour déterminer les lignes de découpage, ont montré des résultats encourageants sur des grilles simples mais insuffisants sur des cas plus complexes. Par exemple, dans des images où certaines lignes sont légèrement courbées ou cassées, notre méthode actuelle a échoué à produire un découpage uniforme.

Nous avons identifié plusieurs axes d'amélioration pour cette fonctionnalité, notamment :

- **Détection adaptative des lignes et colonnes** : Intégrer des algorithmes capables de s'ajuster dynamiquement aux variations de la grille, en détectant les lignes même en cas de distorsion.
- **Normalisation de l'image** : Appliquer des transformations correctives pour redresser la grille avant le découpage, telles que les transformations de perspective ou les corrections de rotation.
- **Validation post-découpage** : Mettre en place une vérification automatique des résultats du découpage pour détecter et corriger les erreurs éventuelles avant de passer à l'étape suivante.

Pour compléter ces améliorations, nous avons également prévu de tester des méthodes inspirées du deep learning, en entraînant un réseau convolutionnel à segmenter directement les images en cases. Ces approches modernes, bien que plus exigeantes en termes de calcul et de données, pourraient offrir une meilleure précision dans des cas complexes.

En conclusion, bien que nous n'ayons pas encore atteint nos objectifs pour cette étape, nous restons optimistes et motivés. Nous croyons fermement que, grâce à un travail acharné, des recherches approfondies et une méthodologie rigoureuse, nous serons en mesure de surmonter ces obstacles et de fournir un système de découpage performant lors de la prochaine soutenance.

6.4 La preuve de développement d'un réseau de neurones

Dans le cadre de ce projet, un réseau de neurones a été conçu, développé et entraîné pour reconnaître des caractères à partir d'images découpées. Ce réseau, basé sur une architecture avec une couche d'entrée, une couche cachée et une couche de sortie, a démontré une capacité satisfaisante d'apprentissage. Les poids et biais des couches ont été ajustés à l'aide de l'algorithme de rétro propagation, et l'entraînement a été réalisé avec succès en utilisant des jeux de données d'images labellisées. Les résultats obtenus montrent que le réseau est capable de converger vers des représentations précises des lettres en fonction des données fournies.

Cependant, malgré ces avancées, nous avons rencontré des limitations dans l'implémentation de la prédiction et son intégration avec la reconstruction directe de la grille et des mots associés. Bien que le réseau soit capable de produire des prédictions pour des images isolées, le lien systématique entre les sorties du réseau et la reconstruction des structures globales (grilles et mots) reste incomplet. Cette difficulté découle principalement de la complexité des interactions entre la gestion des fichiers découpés, le traitement des résultats prédits et leur organisation dans une structure logique.

En conclusion, si la phase d'apprentissage du réseau de neurones constitue une réussite technique et théorique, l'absence d'une intégration complète avec les autres modules du projet (reconstruction de la grille, organisation des mots) limite l'application pratique de ces prédictions. Ce constat met en évidence la nécessité de travailler davantage sur l'interfaçage entre les différents composants pour aboutir à une solution pleinement opérationnelle.

6.5 Interface graphique

L'interface graphique développée dans le cadre de ce projet a été pensée pour simplifier et rationaliser le processus de manipulation des images et d'analyse du contenu. L'objectif était non seulement de fournir un ensemble d'outils techniques mais aussi d'assurer une grande fluidité dans les interactions.

Par exemple, un utilisateur peut importer une image brute, la manipuler grâce à des ajustements successifs (pivotement, amélioration du contraste, réduction de bruit), puis lancer un processus d'OCR pour extraire automatiquement les mots ou lettres détectées. Cette chaîne de traitement, conçue de manière modulaire, rend l'interface adaptable à divers cas d'utilisation. Fonctionnalités Clés et Approches Techniques

6.5.1 Première version

Lors du développement initial de l'interface graphique de l'OCR, nous avons conçu une première version de l'interface graphique. Bien qu'elle ait permis de poser les bases fonctionnelles du projet, cette

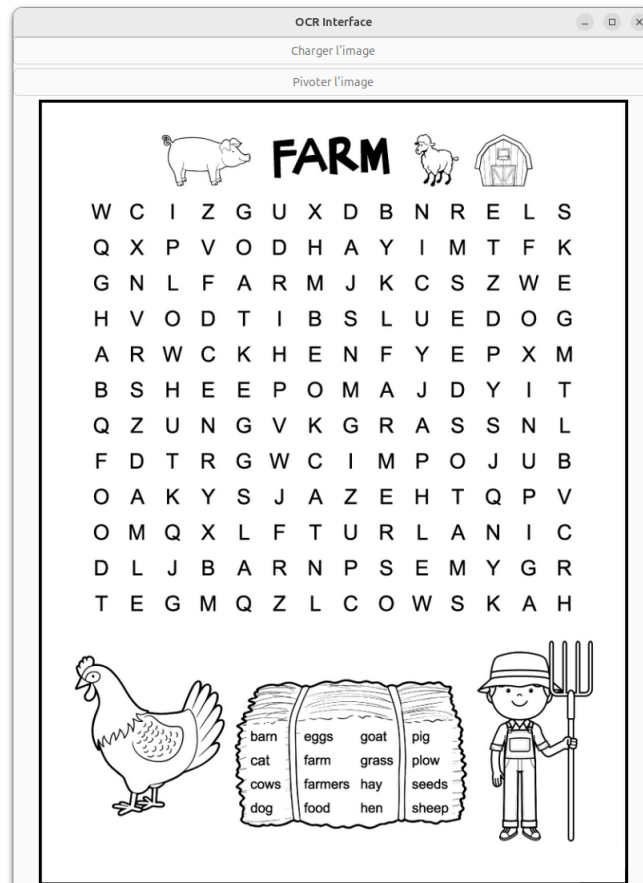


FIGURE 9 – Première version de l'interface graphique.

version présentait de nombreuses limitations, tant sur le plan ergonomique que fonctionnel. Nous avons rapidement identifié des axes d'amélioration essentiels pour répondre aux attentes d'une application pratique et conviviale.

Sur le plan ergonomique, l'interface manquait de structure et de clarté. Les boutons étaient tous regroupés en haut de la fenêtre, ce qui déséquilibrait l'interface visuellement et rendait leur accès moins intuitif. De plus, il n'y avait pas de séparation claire entre les différentes zones de l'application, comme les contrôles et l'affichage principal.

Du point de vue des fonctionnalités, cette première version offrait des options très limitées. L'utilisateur ne pouvait pas enregistrer les modifications apportées à une image, ce qui limitait fortement l'utilité pratique de l'application. Il manquait également un bouton de réinitialisation, obligeant l'utilisateur à redémarrer l'application pour repartir à zéro. Les fonctionnalités avancées, essentielles pour un outil OCR, telles que l'amélioration du contraste, l'inversion des couleurs ou la détection des lettres, étaient complètement absentes à ce stade.

6.5.2 Deuxième version

La deuxième version de notre interface graphique, bien qu'elle ait été enrichie de nombreuses améliorations par rapport à la première, ne satisfaisait pas encore pleinement nos besoins. Elle marque une

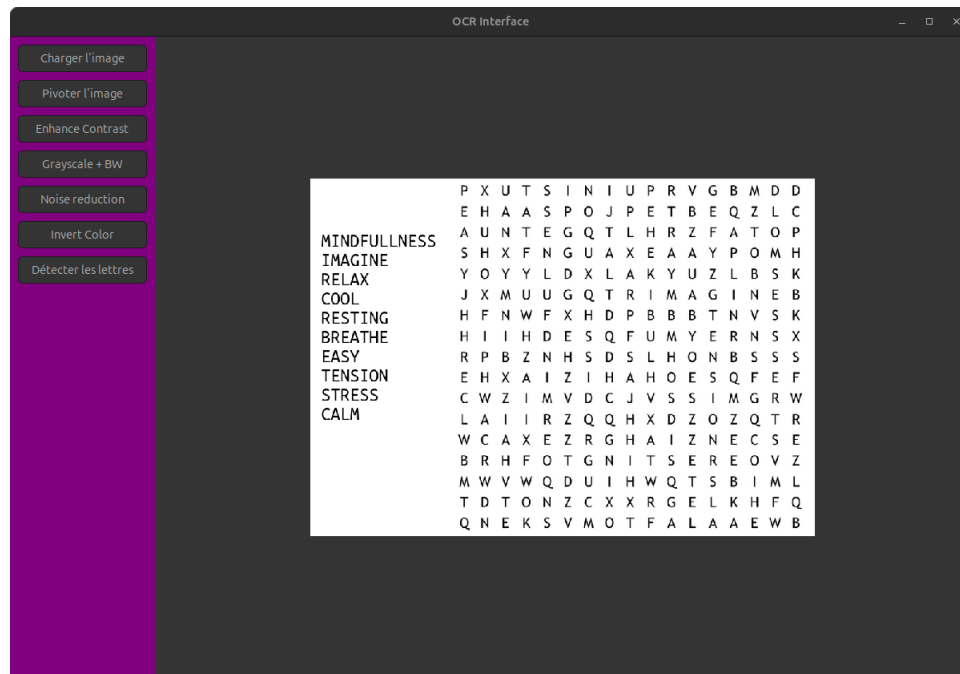


FIGURE 10 – Deuxième version de l'interface graphique.

étape cruciale dans le développement de notre application, mais elle présentait encore des contraintes qui nécessitaient des ajustements supplémentaires.

— Changements par rapport à la version initiale :

Plusieurs des défauts de la première version ont été corrigés dans la deuxième version. En premier lieu, la réorganisation des boutons a favorisé une amélioration de la structure visuelle. Le déplacement des boutons dans une barre latérale à gauche a permis de dégager de l'espace dans la zone centrale afin d'afficher l'image de manière plus claire. En outre, on a ajouté des actions supplémentaires aux boutons, telles que "Améliorer le contraste", "Grayscale + BW", "Réduction du bruit" et "Invert Color", ce qui a étendu les possibilités de manipulation d'image directement dans l'interface.

— Limites persistantes :

Malgré ces avancées, la deuxième version présentait encore plusieurs lacunes. En premier lieu, l'apparence globale de l'application demeurerait simple, voire primitive, et ne donnait pas encore l'impression d'un produit achevé. Malgré leur meilleure position, les boutons demeuraient visuellement homogènes, ce qui rendait l'interface peu attrayante. De plus, des éléments graphiques plus contemporains, tels que des icônes ou des animations, étaient toujours absents pour renforcer l'aspect professionnel de l'application. En ce qui concerne les fonctionnalités, malgré l'intégration de nouveaux outils, certaines fonctionnalités essentielles telles qu'un bouton "Sauvegarder l'image" ou une option "Réinitialiser" pour nettoyer l'interface étaient toujours manquantes.

Elle se divise en plusieurs zones distinctes : une barre de titre, un panneau de contrôle latéral contenant les outils principaux, et une zone centrale dédiée à l'affichage et à l'interaction avec la grille ou l'image chargée.

L'élément central et bien visible est la barre de titre située en haut, affichant clairement le nom de l'application, **SegFaultLess - OCR**. Ce titre précis identifie immédiatement l'application et son

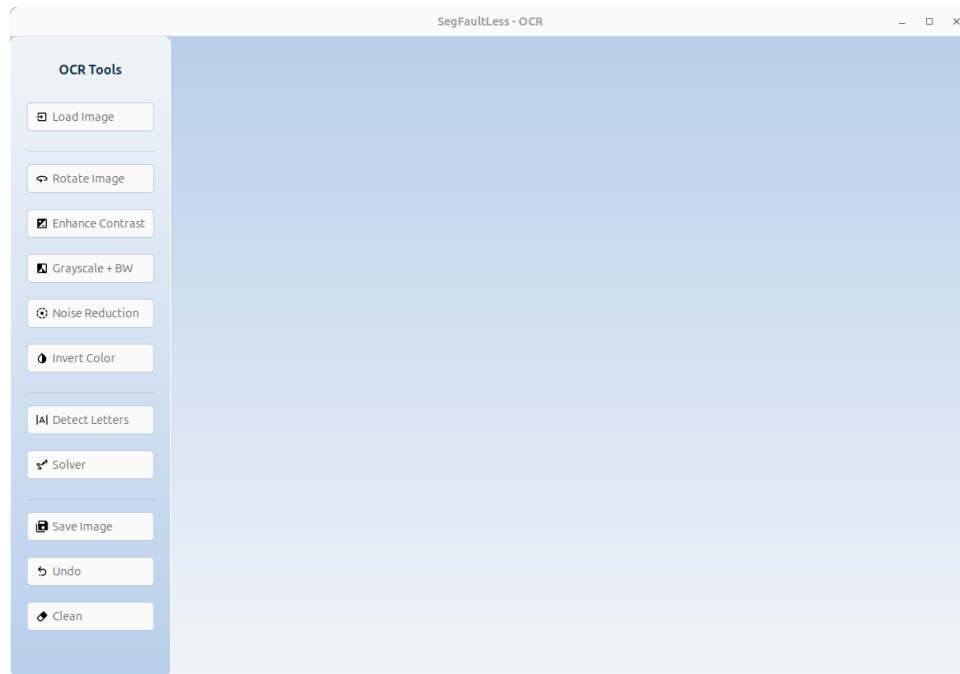


FIGURE 11 – Version finale de l'interface graphique.

domaine d'expertise, offrant une clarté immédiate à l'utilisateur.

Sur la partie gauche de l'écran se trouve un panneau de contrôle vertical qui constitue l'interaction utilisateur. Ce panneau regroupe une série de boutons, chacun correspondant à une action ou à une fonctionnalité spécifique. Parmi ces outils, on trouve des fonctionnalités classiques telles que le chargement d'une image, la rotation, ou encore l'amélioration du contraste. Ces outils sont essentiels pour préparer l'image avant une analyse plus poussée. Par exemple, la possibilité de pivoter une image ou de la convertir en niveaux de gris est particulièrement utile pour s'assurer que les lettres ou éléments textuels sont clairement lisibles avant l'application du Solver.

Le bouton "**Noise Reduction**" se distingue par son utilité pour nettoyer les bruits visuels présents sur les images charge, ce qui est une étape cruciale dans le traitement OCR. De manière similaire, le bouton "**Invert Color**", qui inverse les couleurs de l'image, est particulièrement adapté pour traiter des images contenant du texte blanc sur un fond noir. Une suggestion d'amélioration serait d'afficher un aperçu avant/après pour que l'utilisateur puisse visualiser l'impact de cette réduction, ou de l'application d'un autre filtre.

Un des points centrales de notre interface réside dans son bouton "**Detect Letters**". Cette fonctionnalité représente le cœur de notre application, car elle permet de reconnaître automatiquement les lettres présentes dans l'image. Cela est essentiel pour la résolution de la grille de mots ainsi qu'extraire les lettres ce trouvant dans celui-ci pour les analyse et les résoudre. Un algorithme détecte les lettres présentent dans la grille et les lettres dans la listes de mots a trouve. Elles sont sépare en 2 fichiers pour être résolut.

Le bouton principal de notre interface est le bouton "**Solver**". Après application de multiples filtre et de la détection des lettres, le bouton "**Solver**" peut-être utilisé afin d'afficher le traçage de la résolution de la grille. Ce type de fonctionnalité apporte une grande valeur ajoutée en automatisant des



FIGURE 12 – Detect Letters.

tâches complexes. L'interface pourrait toutefois être améliorée en ajoutant des paramètres configurables pour le solveur, comme la possibilité de choisir les directions de recherche (horizontal, vertical, diagonal).

La zone principale de l'interface, située à droite, est dédiée à l'affichage de la grille ou de l'image importée. Dans notre cas de grille, on affiche une matrice de lettres parfaitement lisible ou non. À côté de cette grille, une liste de mots cibles est affichée, servant de guide pour la recherche. Dans notre exemple ici, les mots listés comme "MINDFULNESS", "RELAX", ou "STRESS", indiquent les mots retrouvés dans la grille après application du prétraitement d'image, de la détection de lettres et du Solver. Cette présentation est claire et intuitive, une amélioration possible est l'interaction directe avec l'image permettant à l'utilisateur de résoudre la grille de mots manuellement et de comparer avec la résolution de notre réseau de neurones.

L'intégration du bouton **"Save Image"** comble une lacune majeure des versions précédentes. Cette fonctionnalité permet désormais aux utilisateurs de sauvegarder les modifications apportées à l'image, rendant l'application bien plus utile dans des scénarios pratiques. L'interface devient ainsi fonctionnellement complète, permettant une manipulation de l'image de bout en bout, depuis son chargement jusqu'à sa sauvegarde. Ce bouton est particulièrement intuitif grâce à son icône de disquette et à son emplacement bien visible dans la barre latérale. Ce détail simple mais essentiel renforce l'efficacité de l'application et la satisfaction des utilisateurs.

Le bouton **"Undo"** complète les fonctionnalités offertes en permettant d'annuler une action récente. On crée une pile qui empile chaque état de l'image, cela est particulièrement utile dans une application impliquant de nombreuses manipulations, où une erreur peut facilement se produire.

Enfin, le bouton **"Clean"** est une autre amélioration significative. Il permet de réinitialiser l'interface en supprimant l'image actuellement affichée et en vidant la pile des actions passées (Undo), sans

avoir besoin de redémarrer l'application. Cela facilite grandement l'expérience utilisateur, notamment dans le cadre d'un usage itératif où des erreurs ou des essais multiples peuvent survenir. Ce bouton apporte également une plus grande flexibilité à l'application, en permettant de repartir facilement sur une nouvelle tâche ou un nouvel ensemble de données.

En résumé, notre interface graphique est bien conçue et offre une base solide pour des applications de manipulation de grilles. Les fonctionnalités essentielles sont présentes, et leur agencement est logique et intuitif. Cependant, des améliorations ergonomiques, comme l'ajout d'options de personnalisation ou de paramètres avancés, pourraient encore enrichir l'expérience utilisateur.

6.6 Tableau d'avancement des tâches

Tâches	Première soutenance	Dernière soutenance
Chargement d'une image	100%	100%
Suppression des couleurs	60%	100%
Rotation de l'image	50%	50%
Prétraitement	Non abordé	100%
Détection de la position grille	X%	90%
Détection de la position de la liste de mots	X%	90%
Détection des cases de la grille	X%	90%
Récupération sous la forme d'image	X%	90%
Réseau de neurones complet et fonctionnel	50%	80%
Résolution de la grille	100%	100%
Affichage de la grille résolue	Non abordé	100%
Sauvegarde de la grille résolue	Non abordé	100%
Interface graphique	Non abordé	85%

TABLE 2 – Tableau d'avancement des tâches

7 Conclusion

Le projet **OCR Word Search Solver**, réalisé par l'équipe SegFaultless, a représenté une expérience d'apprentissage riche et stimulante, alliant théorie et pratique pour répondre à des défis techniques complexes. De la manipulation d'images au développement d'algorithmes de reconnaissance et de résolution, chaque étape a nécessité une collaboration étroite et une mobilisation des compétences individuelles et collectives.

L'un des points forts de ce projet réside dans la mise en œuvre complète et réussie du pipeline de traitement d'images, allant du chargement initial des images jusqu'à leur prétraitement avancé. Ces étapes cruciales ont permis de garantir une qualité et une uniformité optimales des données exploitées par les modules ultérieurs. De plus, la segmentation des images en clusters de lettres, couplée à l'utilisation d'un réseau de neurones pour leur reconnaissance, témoigne de notre capacité à intégrer des approches

modernes de traitement d'image et d'intelligence artificielle.

Cependant, tout au long du projet, des choix stratégiques ont dû être faits en fonction des priorités et des contraintes temporelles. Si le réseau de neurones a montré son efficacité pour l'apprentissage, son intégration complète avec la reconstruction automatique de la grille reste un défi à finaliser. De même, bien que la rotation manuelle des images ait été implémentée, la rotation automatique, initialement prévue, n'a pu être développée faute de temps. Ces limites nous rappellent l'importance d'une planification rigoureuse et d'une gestion efficace des ressources dans des projets ambitieux.

En parallèle, l'élaboration d'une interface graphique fonctionnelle a permis de proposer une expérience utilisateur fluide et intuitive, consolidant l'ensemble des fonctionnalités en une application pratique et exploitable. Cette interface constitue une base solide pour des améliorations futures, telles que l'intégration de fonctionnalités avancées comme l'édition manuelle des grilles ou l'interaction directe avec les résultats.

En conclusion, ce projet a permis à notre équipe de relever de nombreux défis techniques et organisationnels, tout en renforçant nos compétences en traitement d'images, en développement d'algorithmes, et en collaboration sur un projet de grande envergure. Bien que certaines fonctionnalités restent à affiner ou à compléter, nous sommes fiers du chemin parcouru et des résultats obtenus. Ce projet constitue une fondation prometteuse pour des développements ultérieurs, et il a enrichi chacun des membres de l'équipe d'une expérience précieuse pour leurs parcours professionnels futurs.

Nous souhaitons remercier notre encadrement pour le soutien et les conseils prodigués tout au long de cette aventure. Ce projet nous a non seulement permis de mettre en pratique nos acquis, mais également d'explorer des domaines techniques passionnants et de collaborer avec une équipe dynamique et motivée.

L'OCR Word Search Solver est plus qu'un aboutissement technique : il est le témoignage de notre engagement, de notre apprentissage, et de notre capacité à transformer des idées complexes en réalisations concrètes.

Petit conseil en OCR : si vous commencez à voir des caractères bizarres, prenez une pause ... ou vérifiez votre prétraitement !

A Annexes

Voici l'arborescence du projet OCR, détaillant les fichiers et dossiers pour les différents modules :

```
→ OCR tree
├── Makefile
├── Makefile_neural
├── Makefile_solver
├── bin
│   └── image_loader
├── data
│   └── examples
│       └── level1.png
├── grid
├── include
│   ├── load_image.h
│   ├── neural_net.h
│   ├── preprocessor.h
│   └── solver.h
├── neural_test
├── obj
│   ├── image_processing.o
│   ├── load_image.o
│   ├── main.o
│   ├── main_neural_net.o
│   ├── main_solver.o
│   ├── neural_net.o
│   └── solver.o
├── solver
└── src
    ├── image_processing.c
    ├── image_processing.o
    ├── load_image.c
    ├── load_image.o
    ├── main.c
    ├── main.o
    ├── main_neural_net.c
    ├── main_neural_net.o
    ├── main_solver.c
    ├── main_solver.o
    ├── neural_net.c
    └── neural_net.o
```

FIGURE 13 – Arborescence du projet OCR

```
→ OCR ./solver grid HORIZONTAL
(0,0)(9,0)
→ OCR ./solver grid EPITA
Not Found
→ OCR ./solver grid OCR
Not Found
→ OCR cat grid
HORIZONTAL
DXRAHCLBGA
DIKCILEOKC
IGAJHYLYHI
HGFGODTIOT
GDLROWKBFR
PLNRDNERGE
JHAIDUAJGV
UKGFFOLLEH
```

FIGURE 14 – Preuve de la résolution de la grille à partir d'un fichier

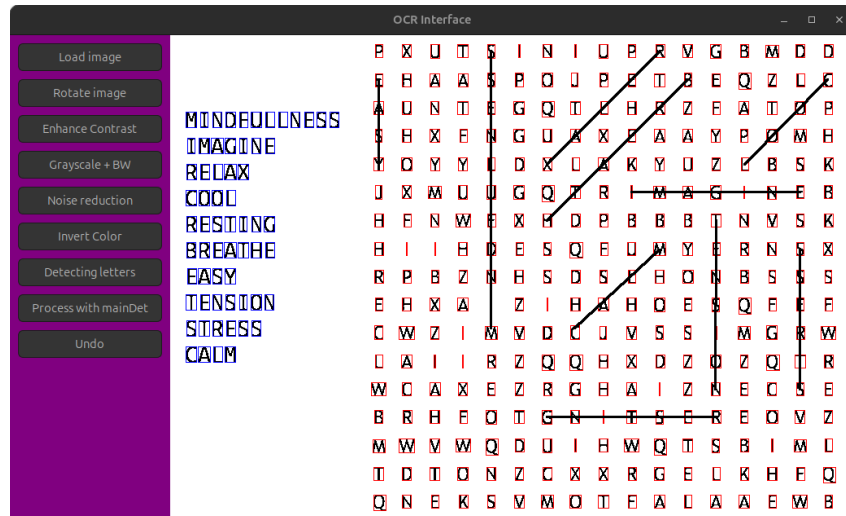


FIGURE 15 – Preuve de la résolution de la grille à partir de l'interface graphique

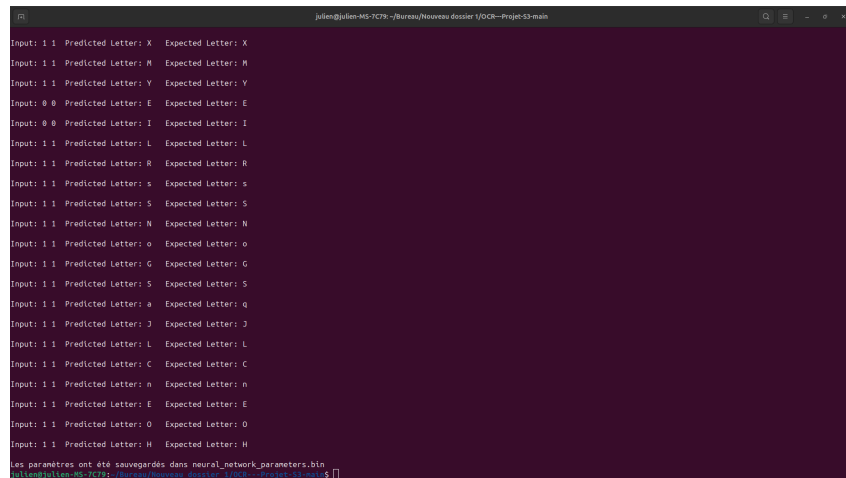


FIGURE 16 – Preuve de la phase d'apprentissage du réseau de neurones